Finding the Spaghetti Nine while Curating MNIST: How Specialist Models Reveal Hidden Dataset Quality Issues

Author: Antonio Rueda-Toicen

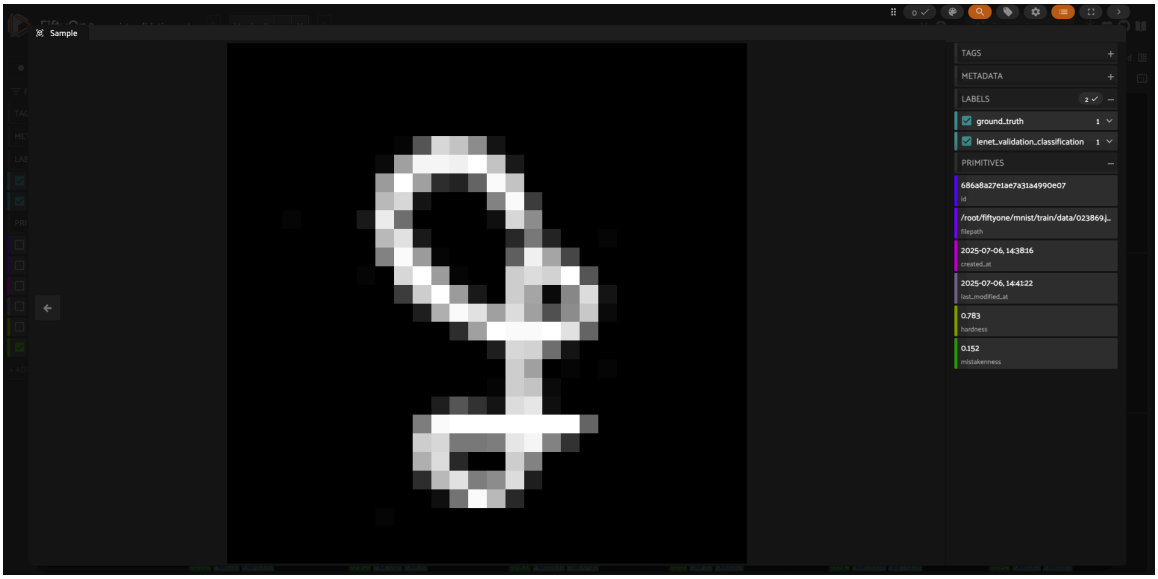

Figure: The Spaghetti Nine Lurking Inside MNIST

What if one of the world's most used machine learning datasets isn't as clean as we thought? In this second part of our series, we turn a specialist LeNet-5 model into a data detective, uncovering the infamous "spaghetti nine" and other quality issues hiding in plain sight within the canonical MNIST dataset.

Introduction

The MNIST dataset is the "hello world" of computer vision, a benchmark used by millions of practitioners worldwide. For over two decades, many practitioners have treated this collection of 70,000 handwritten digits as a gold standard of clean, reliable training data. But what if this assumption is wrong?

Who This Article Is For: Data scientists, machine learning engineers, and researchers focused on data quality. This post provides a practical guide for using model predictions to audit and improve dataset quality, offering techniques that extend far beyond MNIST to real-world data challenges.

What You'll Learn:

- How to build and (re)train the classic LeNet-5 model from scratch using PyTorch and integrate its predictions to a FiftyOne dataset
- The power of FiftyOne's "uniqueness", "mistakenness", and "hardness" metrics for identifying interesting and problematic samples
- A systematic approach to dataset curation using FiftyOne Brain
- How to discover and handle ambiguous samples
- Practical strategies for data augmentation and model retraining

From Foundation Models to Specialists: The Performance Leap

In a previous blogpost, we established our baseline with CLIP's zero-shot performance on MNIST. While this demonstrated the remarkable generalization capabilities of foundation models, it also highlighted their limitations in specialized domains. We introduced the classic LeNet-5 and showed how it's able to produce near perfectly accurate predictions on our dataset.

Here's where our story takes an introspective turn. Instead of just achieving higher accuracy, our specialist LeNet-5 model has become a powerful tool for data auditing, revealing that even the most canonical dataset in machine learning contains ambiguous samples, labeling errors, and edge cases that might have gone unnoticed before. The key insight is that **a well-trained model that disagrees with ground truth labels often reveals annotation errors rather than model failures**. When a network trained on 50,000+ examples predicts a different label than the ground truth with high confidence, we should investigate the data, not just blame the model and keep hacking it to increase its accuracy score.

The LeNet-5 Advantage

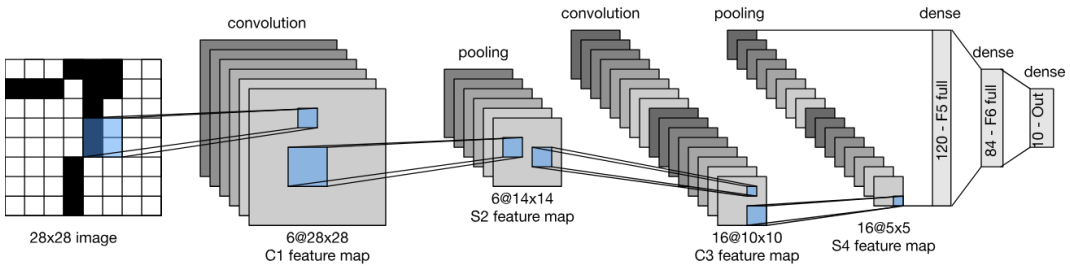

Figure: LetNet-5 architecture overview

LeNet-5, proposed by Yann LeCun in 1998, remains one of the most elegant demonstrations of convolutional neural network principles. Its hierarchical architecture learns increasingly complex features: edges and curves in early layers, then digit-specific patterns in deeper layers. This learned representation becomes our lens for examining data quality.

Building LeNet-5: The Specialist Architecture

LeNet-5's architecture is simple yet powerful for digit recognition. The network consists of alternating convolutional and pooling layers for feature extraction, followed by fully connected layers for classification.

Architecture Implementation

```
class ModernLeNet5(nn.Module):
    """
    Modernized version of LeNet-5 with ReLU activations and max pooling.
    Often performs better on MNIST than the original version.
    """

    def __init__(self, num_classes=10):
        super(ModernLeNet5, self).__init__()

        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
        self.conv3 = nn.Conv2d(16, 120, kernel_size=4)

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(120, 84)
        self.fc2 = nn.Linear(84, num_classes)

        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = F.relu(self.conv3(x))

        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)

        return x
```

Training for Reproducible Results

Reproducibility becomes critical when using models for data auditing. We ensure consistent results across runs by setting seeds for all random operations:

```
def set_seeds(seed=51):
    """Set seeds for complete reproducibility across all libraries."""
    os.environ['PYTHONHASHSEED'] = str(seed)
    os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':4096:8'

    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False
```

From FiftyOne to PyTorch: The Data Pipeline

FiftyOne's rich dataset objects need transformation into PyTorch-compatible formats for training:

```
class CustomTorchImageDataset(torch.utils.data.Dataset):
    def __init__(self, fiftyone_dataset, image_transforms=None, label_map=None):
        self.image_paths = fiftyone_dataset.values("filepath")
        self.str_labels = fiftyone_dataset.values("ground_truth.label")
        self.image_transforms = image_transforms
        self.label_map = label_map or {str(i): i for i in range(10)}

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert('L')
        if self.image_transforms:
            image = self.image_transforms(image)

        label_str = self.str_labels[idx]
        label_idx = self.label_map[label_str]

        return image, torch.tensor(label_idx, dtype=torch.long)
```

Achieving Specialist Performance

After training for 10 epochs with careful validation monitoring, our LeNet-5 model achieves >99.8% accuracy on the test set, demonstrating the power of task-specific optimization.

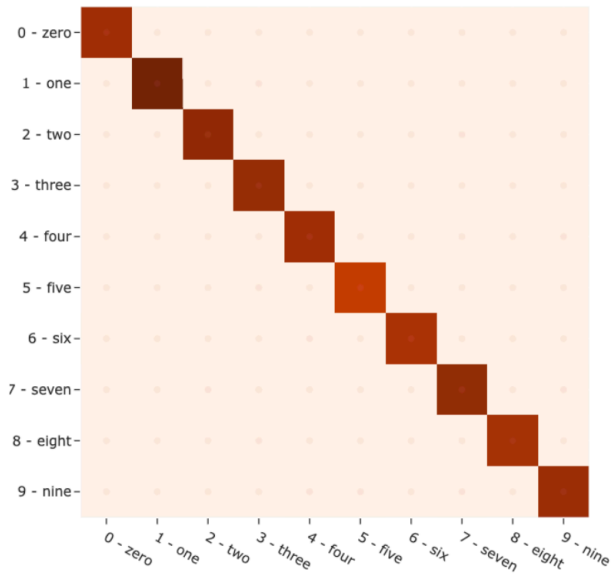

Figure: Confusion Matrix for MNIST's Test Set

But accuracy scores and confusion matrices tell only part of the story. The real value lies in what the model learned and where it disagrees with our labels and expectations. We need to look deeper into the model's understanding of our data.

The Real Power: Using Models to Audit Data

This is where our analysis becomes powerful. Instead of celebrating high accuracy and moving on, we turn our trained model into a data detective, using its learned representations to examine the quality of the dataset itself.

Understanding Hardness and Mistakenness

FiftyOne Brain provides two very useful metrics for data quality assessment:

Hardness quantifies how difficult it is for a model to correctly classify a given sample. Specifically, FiftyOne's hardness method computes the entropy of the predicted class probabilities for each sample. Entropy, in this context, measures the uncertainty in the model's predictions:

- **Low entropy (low hardness):** The model is confident in its prediction for that sample.
- **High entropy (high hardness):** The model is uncertain, indicating the sample is harder for the model to classify.

This metric is useful for identifying samples that are challenging for your model, allowing you to focus training or analysis on these "hard" samples. The computed hardness value is stored as a scalar field (typically named `hardness`) on each sample in your dataset. You can then use this field to sort, filter, or further analyze your data, such as finding the hardest false positives or negatives in your validation set.

Mistakenness quantifies the likelihood that a ground truth annotation is incorrect. It is computed by comparing your dataset's ground truth annotations to model predictions. The FiftyOne Brain's `compute_mistakenness()` method assigns a mistakenness score (a float value) to each object or sample, indicating how likely it is that the annotation is a mistake.

- High mistakenness values suggest that the annotation may be wrong or questionable, such as a mislabeled object.

Mistakenness is used to help you identify and prioritize potential annotation errors for review and correction, improving the quality of your dataset and your model.

```
# Compute hardness based on prediction uncertainty
fob.compute_hardness(test_dataset, label_field='lenet_classification')

# Compute mistakenness to identify potential label errors
fob.compute_mistakenness(test_dataset,
```

```
    pred_field="lenet_classification",
    label_field="ground_truth")
```

The Audit Process: Turning Models Against Their Training Data

The most powerful exploration comes when we apply our trained model back to its own training data. This creates a feedback loop where the model's learned understanding becomes a quality control mechanism for the data that taught it.

```
# Apply the trained model to training data for audit
train_dataset.apply_model(
    model=trained_lenet_model,
    label_field="lenet_train_classification",
    store_logits=True
)

# Compute quality metrics on training data
fob.compute_mistakenness(train_dataset,
                         pred_field="lenet_train_classification",
                         label_field="ground_truth")

# Find the most suspicious samples
mistakenness_quantiles = train_dataset.quantiles("mistakenness", [0.99])
suspicious_samples = train_dataset.match(
    F("mistakenness") > mistakenness_quantiles[0]
).sort_by("mistakenness", reverse=True)
```

This approach transforms the overwhelming task of reviewing 60,000 training images by hand into a focused investigation of a few dozen suspicious samples.

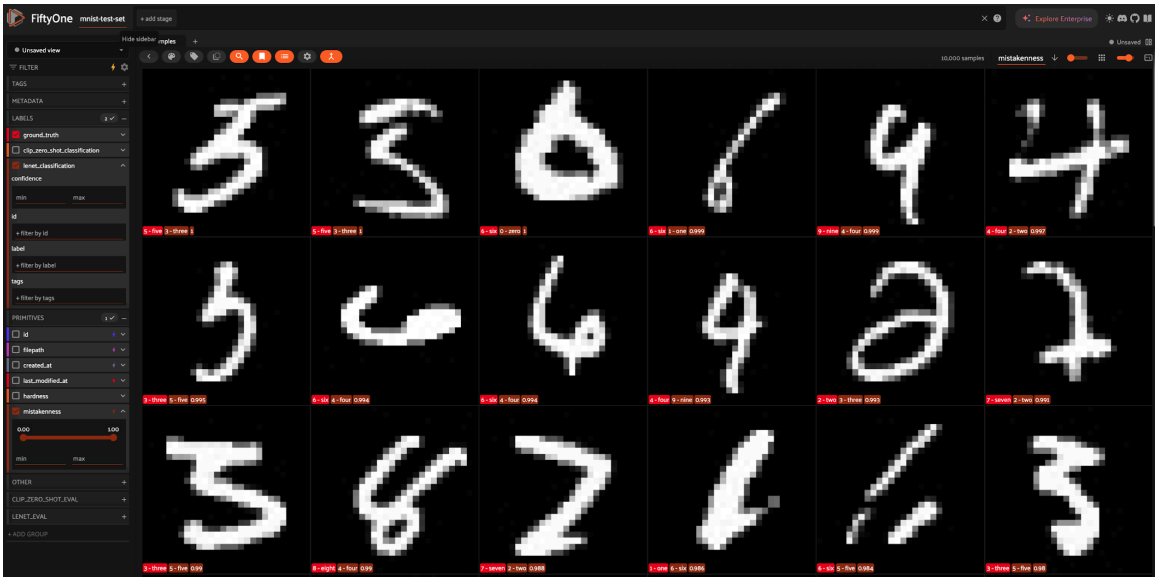

Figure: Decreasing Mistakenness on MNIST's test set

Systematic reduction of search space from thousands of images to the most problematic samples

Discovering the Needles in the Haystack

Using our metrics, we've reduced the search space from 60,000 training samples to just a handful of suspicious cases. This is where we make our most interesting discoveries.

The Infamous "Spaghetti Nine"

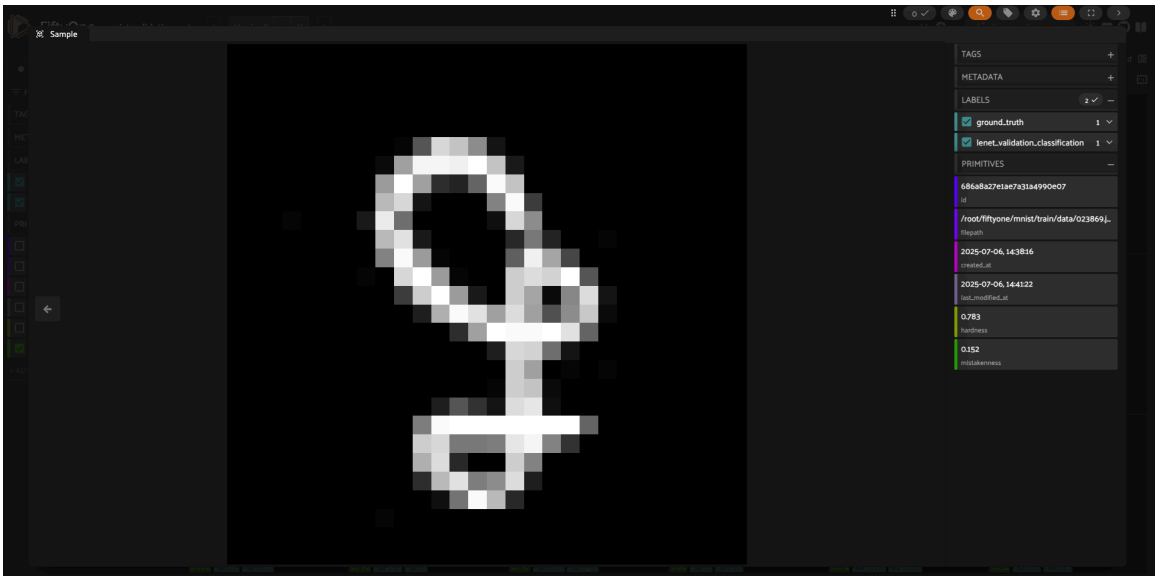

Figure: The Spaghetti Nine a sample in the training set with the ground truth label “9”

Our analysis on the training set surfaces the infamous "spaghetti nine," a sample with ground truth label “9” that has puzzled practitioners for years. This image demonstrates why high mistakenness scores often indicate genuine labeling ambiguities rather than model failures. The handwritten digit exhibits characteristics that could be interpreted as either a "9" or other digits, creating uncertainty that no amount of training can resolve.

Other Interesting/Problematic Discoveries

The audit reveals a pattern of quality issues throughout the dataset:

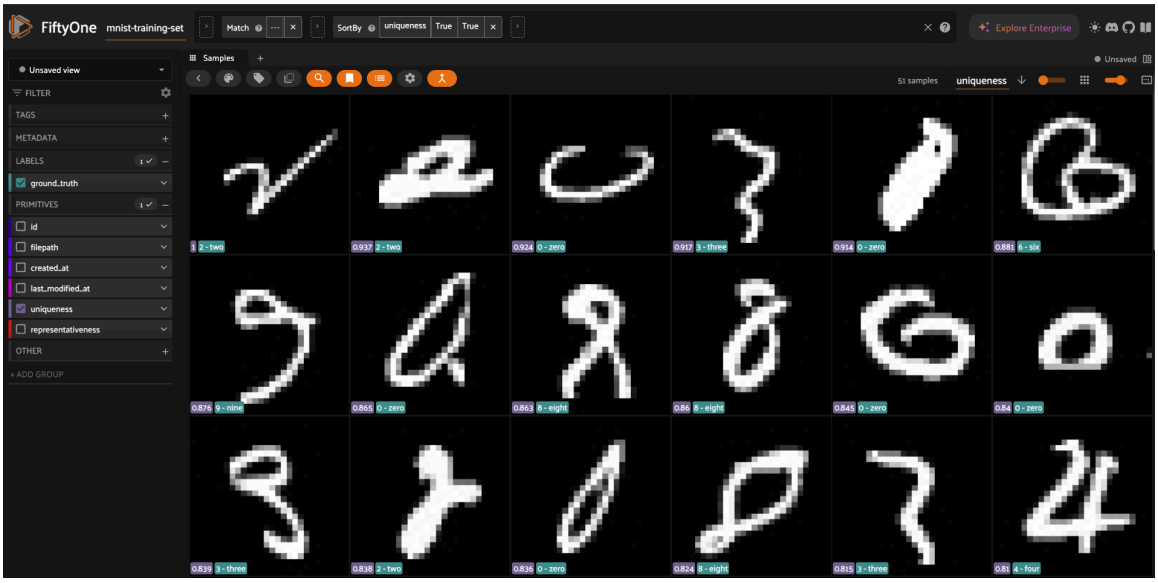

Figure: Samples sorted by decreasing Uniqueness score

Collection of ambiguous samples discovered through systematic mistakenness analysis

Digit Ambiguities: Samples where stroke patterns create genuine uncertainty between classes (4 vs 9, 1 vs 7, 3 vs 8).

Annotation Errors: Cases where the ground truth label appears incorrect based on visual inspection and model confidence.

Edge Cases: Unusual handwriting styles or artifacts that fall outside normal digit variations.

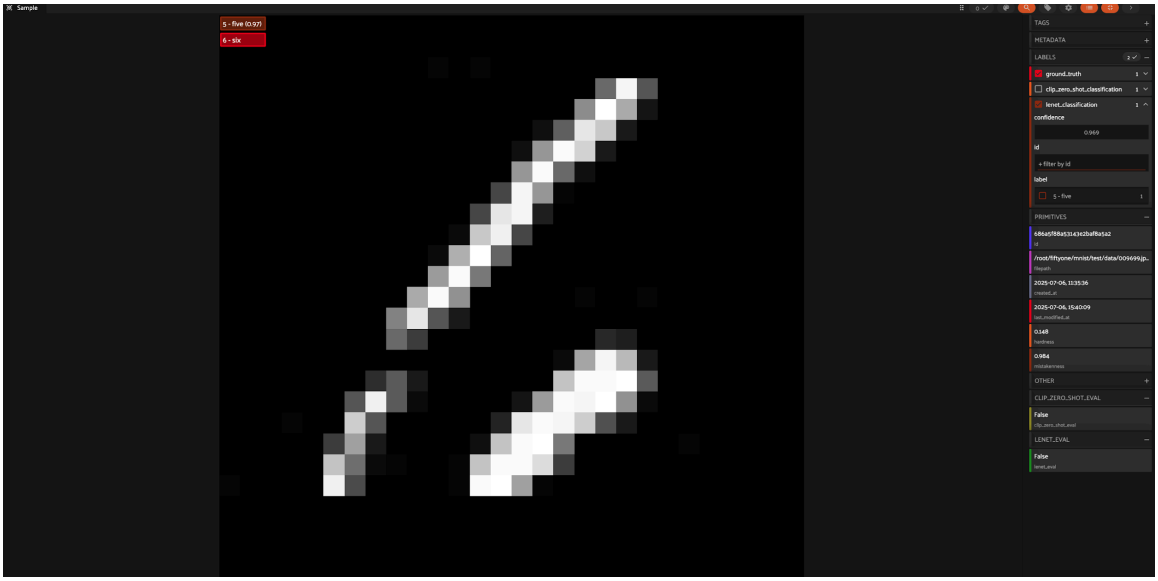

Figure: Weird Test Digit

Example of an edge case that challenges both human annotators and machine learning models, one that we should consider for removal from a curated version of the dataset

Visualizing the Model's Understanding with Embeddings

A powerful way to understand how our specialist model perceives the dataset is to visualize its embeddings. As we did with CLIP before, we can extract the feature vectors (embeddings) from the penultimate layer of our LeNet-5 model for each image. These high-dimensional vectors represent the model's rich, learned understanding of each digit.

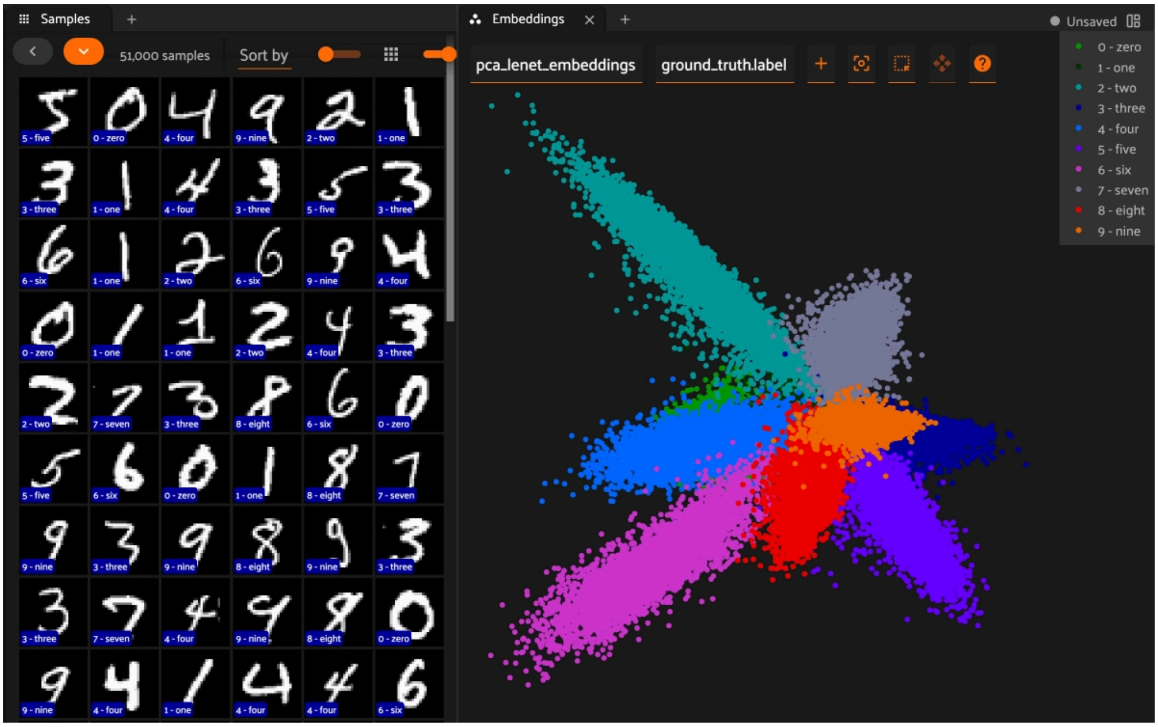

Figure: PCA Projection of LeNet-5 Embeddings on the Training Set

To make these embeddings viewable, we can use a dimensionality reduction technique like Principal Component Analysis (PCA) to project them down into a 2D space. The result is a stunning visualization of the model's "mental map" of the MNIST dataset. The Lasso tool in the FiftyOne's app panel allows us to select regions of the embedding space and inspect them closer. Take some

This embedding plot is more than just a pretty picture. It's an interactive tool for data exploration. We can see clear, well-separated clusters for each digit, which visually confirms why our model is so accurate. We can also spot potential areas of confusion where clusters are close together (like the 4s, 7s, and 9s) or identify outliers that sit far from their respective clusters. By selecting a group of points in the embedding space, we can immediately view the corresponding images, allowing us to navigate the dataset not by random chance, but through the lens of our model's learned knowledge. This powerful

visualization sets the foundation for our next step: using the model's embedding space to audit our data quality.

The Unique Six Discovery

Beyond mistakenness, we can also search for statistical outliers by analyzing the uniqueness of each sample. The uniqueness score in FiftyOne is computed by measuring how dissimilar each sample is from its nearest neighbors in an embedding space. The algorithm finds each sample's three nearest neighbors, weights their distances (60%, 30%, 10%), and normalizes these weighted distances to produce scores between 0 and 1. Higher scores indicate samples that are more isolated or distinct in the embedding space, while lower scores indicate samples with many close neighbors. The most unique sample in the dataset receives a score of 1.

We use the embeddings generated from the penultimate layer of our specialist LeNet-5 model. This means we are measuring uniqueness not just based on raw pixel values, but on the rich, learned features our model uses to distinguish between digits. This approach surfaces samples that the model finds unusual, which is a powerful signal for curation.

```
# Compute uniqueness based on these specialist embeddings

fob.compute_uniqueness(

    train_dataset,

    embeddings_field="lenet_embeddings"

)


# Find and view the most unique samples

unique_samples_view = train_dataset.sort_by("uniqueness", reverse=True)

session.view = unique_samples_view
```

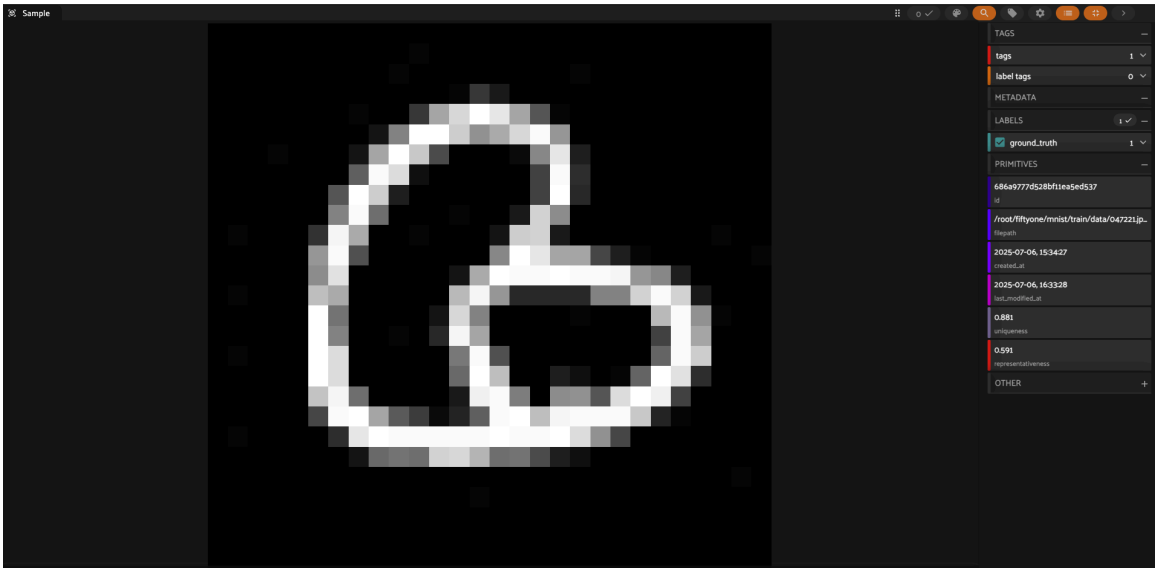

Figure: A unique six, should we keep it in the dataset or remove it?

A highly unique sample that stands apart from typical "6" representations in the embedding space

Our uniqueness analysis reveals samples that are statistical outliers within their class. This "unique six" demonstrates handwriting that, while correct, differs from typical representations. Such samples can skew model training and deserve special attention during dataset curation.

Search for Statistical Outliers

We can also use statistics to help our search. By using quantiles and systematic filtering, we surface the most problematic samples without manual review of the entire dataset:

```
# Focus on the intersection of difficult and misclassified samples
highly_mistaken = train_dataset.match(F("mistakenness") > 0.999)
highly_hard = train_dataset.match(F("hardness") > 0.999)
problematic_samples = train_dataset.select(
    set(highly_mistaken.values("id")).intersection(
        set(highly_hard.values("id"))
    )
)
```

This approach enables us to find the needles in the haystack, surfacing quality issues that would be impossible to identify through manual inspection alone. I encourage you to complement this with searches through the similarity index that we have created using the LeNet-5 embeddings in the accompanying notebook.

Dataset Curation Strategies in Action

Once we've identified problematic samples, we face strategic decisions about how to handle them. Our approach balances data quality improvement with model robustness.

The Remove vs Augment Decision Framework

Deciding whether to remove or augment a problematic sample involves a strategic choice. Removal is appropriate for samples with definite annotation errors that would otherwise mislead the training process. This action is also warranted when a sample's ambiguity is so pronounced that human experts cannot agree on a label, or when an edge case is the result of a scanning artifact rather than a genuine example of handwriting. Conversely, augmentation is the preferred strategy for samples that represent authentic but uncommon variations, such as rare handwriting styles. Augmentation also applies to edge cases that reflect actual data diversity and for high-quality, correctly labeled samples where the model shows low confidence. This approach helps the model generalize better to infrequent examples.

Removing too many unique but valid samples might bias the model against rare handwriting styles, while augmenting them can help it generalize better. Of course, personal judgment comes into play while curating questionable samples. For example, I would remove the Spaghetti Nine from the dataset and augment several of the highly unique samples that are clearly distinguishable as numbers. With FiftyOne we can tag both of these cases for iterative exploration.

The IDK Alternative: For highly ambiguous samples, consider creating an "I Don't Know" class rather than forcing binary decisions. This approach acknowledges uncertainty rather than hiding it within existing classes. As a follow up, try creating a binary classifier for this IDK label or adding it as the 11th class of a new LeNet-5 variation.

Targeted Data Augmentation

Instead of generic augmentation, we focus on the specific samples our model struggles with:

```
# Create augmented versions of problematic samples
augmented_dataset = AugmentedMNISTDataset(
    problematic_samples,
    label_map=label_map,
    base_transforms=image_transforms,
    augmentations=mnist_augmentations,
    augment_factor=9  # Create 9 variations per problematic sample
)

# Combine with cleaned training data
combined_dataset = ConcatDataset([cleaned_train_set, augmented_dataset])
```

MNIST-Appropriate Augmentations:

- Small rotations (±10-15 degrees): Natural handwriting orientation variations
- Small translations (±2-3 pixels): Centering variations
- Elastic deformations: Simulate different handwriting styles
- Avoid heavy distortions that could make digits ambiguous

Curation Impact Measurement

We track the impact of our curation decisions through systematic comparison:

```
# Create views for analysis
originally_wrong = test_dataset.match(
    F("lenet_classification.label") != F("ground_truth.label")
)
now_correct = originally_wrong.match(
    F("retrained_lenet_classification.label") == F("ground_truth.label")
)

print(f"Samples fixed by curation: {len(now_correct)}")
```

This measurement framework ensures our curation efforts produce measurable improvements.

Retraining and Measuring Impact

The proof of effective curation lies in improved model performance. We retrain our LeNet-5 model using the curated dataset and measure the impact through systematic analysis.

Fine-tuning Strategy

Rather than training from scratch, we fine-tune our existing model with the curated data:

```
# Load best model for retraining
retrain_model = ModernLeNet5()
retrain_model.load_state_dict(torch.load(best_model_path))
retrain_model = retrain_model.to(device)

# Use lower learning rate for fine-tuning
retrain_optimizer = Adam(retrain_model.parameters(),
                         lr=0.0001,  # 30x smaller than initial training
                         weight_decay=1e-3)  # Stronger regularization
```

Performance Comparison

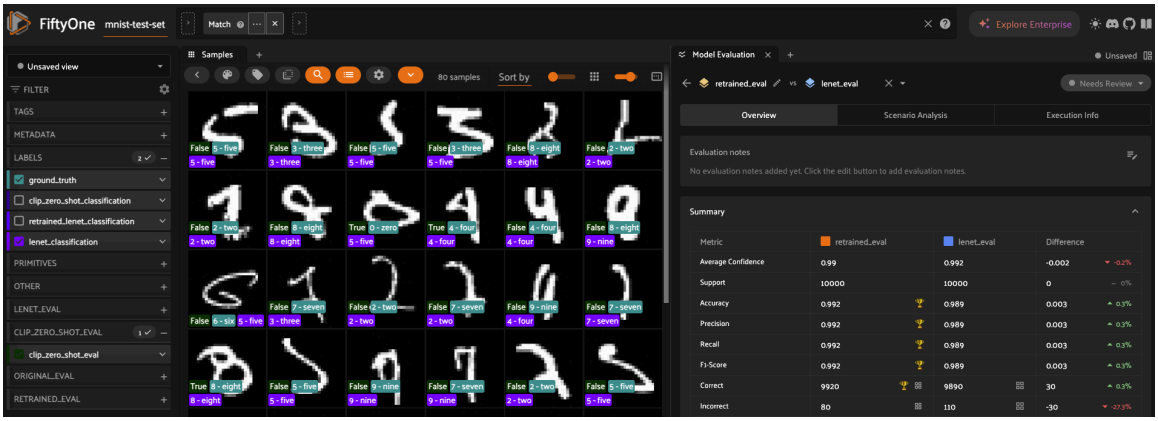

Figure: Retrained Evaluation Comparison

Before and after comparison showing the impact of dataset curation on model performance

The systematic comparison reveals several improvements:

Accuracy Gains: While absolute accuracy improvement may be modest (from 99.8% to 99.9%), the reduction in confidence on incorrect predictions is significant.

Error Type Changes: Curation shifts error patterns away from labeling ambiguities toward genuine recognition challenges.

Confidence Calibration: The retrained model shows better calibrated confidence scores, with higher confidence on correct predictions and appropriate uncertainty on ambiguous cases.

Samples Fixed by Curation

Again, there will be a possible disconnect between what we see in our metrics for net improvement and the samples that we have 'fixed'. Let's take a closer look.

The Net Improvement Analysis

```
# Calculate net improvement
samples_fixed = len(now_correct)
samples_broken = len(now_wrong)
net_improvement = samples_fixed - samples_broken

print(f"Net improvement: {net_improvement} correctly classified samples")

## Output: 0.0041 (+41 samples fixed by retraining)
```

Our metrics showed that we have now +41 new samples being correctly labeled due to our retraining. Let's take a closer look.

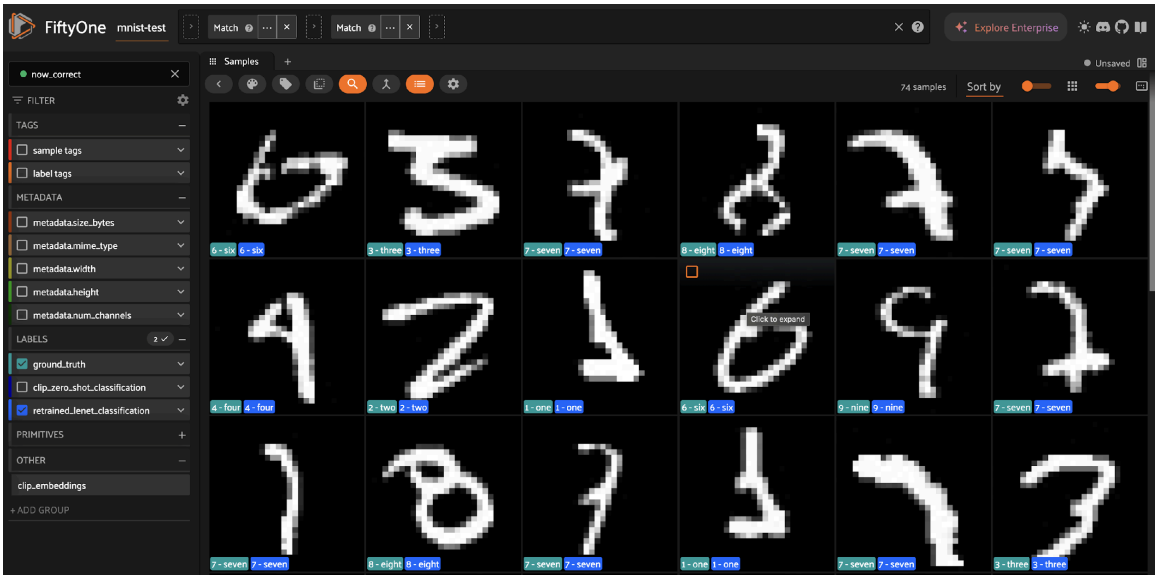

Figure: Now 'Correct' View

Examples of samples that were misclassified before curation but correctly classified after retraining with curated data. Do we still want them or should consider them as candidates for deletion?

The "now correct" view showcases samples where our retraining efforts paid off. These examples validate our approach by showing measurable improvements on problematic cases. However, notice that many of these edge cases are interesting cases for data augmentation while others are **still questionable.** Should we keep them or remove them from the dataset? Well, **that's up to human judgment**. I encourage you to try using the similarity index computed with the fine-tuned LeNet-5 embeddings to find images close to the ones that we have identified below.

Let's label everything that we want to remove and push the data and all our curation work to HuggingFace for peers to review our work.

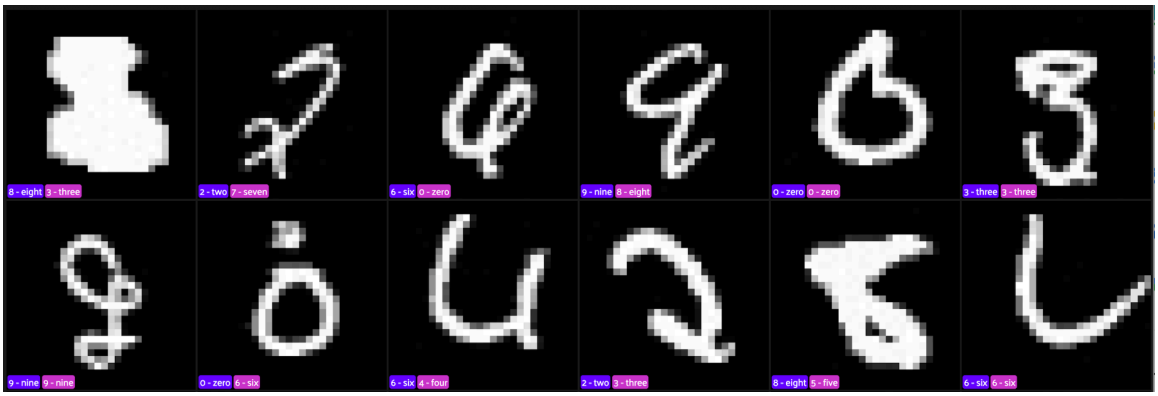

Figure: Samples marked for removal

```
# Create a new merged dataset to hold everything

merged_dataset = fo.Dataset("curated-mnist")


# Add samples from each split, tagging them appropriately

for sample in train_dataset:

    sample.tags.append("train")

    merged_dataset.add_sample(sample)


for sample in val_dataset:
```

```
    sample.tags.append("validation")
    merged_dataset.add_sample(sample)

for sample in test_dataset:
    sample.tags.append("test")
    merged_dataset.add_sample(sample)

# Make the dataset persistent
merged_dataset.persistent = True

# Define an export directory and save the dataset
export_dir = Path.cwd() / "curated_mnist_fiftyone"
merged_dataset.export(
    export_dir=str(export_dir),
    dataset_type=fo.types.FiftyOneDataset,
    export_media=True  # Save image files along with metadata
)

print(f"Merged and curated dataset saved to: {export_dir}")
```

This will create a FiftyOne dataset with all the curation work that we have done so far on your own HuggingFace account. You can get the full output of our work here: https://huggingface.co/datasets/Voxel51/curated-mnist

The Broader Implications: Rethinking Dataset Quality

Our MNIST investigation reveals implications that extend far beyond handwritten digits, challenging fundamental assumptions about dataset reliability in machine learning.

The Myth of Perfect Datasets

The discovery of quality issues in MNIST, one of the most scrutinized datasets in machine learning, demonstrates that **no dataset is immune to quality problems**. If MNIST contains

labeling errors and ambiguous samples, what does this mean for less examined datasets used in production systems?

The Kaggle Reality Check: Examination of MNIST digit recognition competition leaderboards reveals participants claiming perfect 1.0 accuracy. Our analysis suggests these scores may indicate data leakage rather than algorithmic cleverness. The corrupted samples we discovered make perfect accuracy impossible without either removing problematic samples (impossible on the Kaggle platform) or accessing test set information during training (the full MNIST dataset is widely available) .

Data-Centric AI Principles

Our investigation demonstrates core principles of data-centric artificial intelligence:

Models as Data Auditors: Well-trained models can serve as sophisticated quality control mechanisms, identifying annotation errors and edge cases that human reviewers might miss.

Systematic Quality Assessment: Statistical approaches using metrics like mistakenness and hardness scale quality assessment from manual inspection to automated screening.

Iterative Curation: Dataset quality improvement should be treated as an ongoing process rather than a one-time cleaning exercise.

Quality Impact Measurement: Curation efforts must be validated through measurable performance improvements, not just subjective quality assessments.

Beyond MNIST: Real-World Applications

The techniques demonstrated here apply to real-world data challenges:

Medical Imaging: Using specialist models to identify mislabeled medical scans or ambiguous cases requiring expert review.

Industrial Inspection: Applying trained defect detection models to audit their own training data, identifying mislabeled quality control images.

Autonomous Systems: Using perception models to identify problematic samples in driving or robotics datasets where annotation errors could have safety implications.

Practical Implementation Guide

Here's how to apply these dataset curation techniques to your own projects:

Step 1: Train Your Specialist Model

Start with a model optimized for your specific task. The better the model's performance, the more reliable its judgments about data quality.

Step 2: Apply Model to Its Own Training Data

Turn your trained model into a data auditor by applying it to the dataset that trained it.

Step 3: Compute Quality Metrics

Use FiftyOne Brain to compute uniqueness, hardness and mistakenness metrics.

Step 4: Surface Problematic Samples

Use statistical filtering and similarity indices produced with embeddings to focus on the most suspicious cases.

Step 5: Make Curation Decisions

For each suspicious sample, decide whether to remove, augment, or reclassify.

Step 6: Measure Impact

Validate curation effectiveness through retraining and performance comparison.

Best Practices and Pitfalls

Do:

- Maintain reproducible experiments through proper seed setting
- Focus manual review on problematic samples identified through statistical methods
- Measure curation impact through quantitative performance metrics
- Document curation decisions for future reference
- Consider domain expertise when making removal vs augmentation decisions

Don't:

- Remove samples based only on model disagreement without human review
- Apply aggressive augmentation that could distort the underlying data distribution
- Skip impact measurement and validation
- Apply curation techniques without consideration across different domains

When to Apply Dataset Curation

High Priority Scenarios:

- Safety-critical applications where annotation errors have serious consequences
- Cases with known annotation challenges or multiple annotator disagreement
- Datasets showing poor model performance despite apparent simplicity
- Production systems where model confidence calibration is crucial

Consider Carefully:

- Research benchmarks where maintaining dataset integrity is important for comparison
- Cases where removing samples might introduce bias or reduce diversity
- Situations where the cost of manual review exceeds the benefit of improvement

Conclusion: The Future of Data-Centric AI

Our journey from CLIP's zero-shot performance evaluation to LeNet-5's 99.9%+ specialist accuracy revealed something more valuable than improved metrics: a systematic approach to dataset quality assessment that scales beyond manual inspection.

The discovery of the "spaghetti nine" and other problematic samples in MNIST demonstrates that even the most trusted datasets require ongoing quality attention. It also shows how specialist models can serve as sophisticated data auditors, identifying quality issues that would be impossible to find through traditional approaches.

Key Takeaways:

- Dataset quality often matters more than model architecture for achieving reliable performance
- Models can audit their own training data when used with systematic approaches
- Statistical approaches scale quality assessment from manual inspection to automated screening
- Even canonical datasets need curation as our understanding and standards evolve

The techniques demonstrated here extend far beyond MNIST to any supervised learning scenario where data quality impacts model reliability. As machine learning systems move into critical applications, the ability to assess and improve dataset quality through systematic approaches becomes not just useful, but essential.

The future of AI lies not just in building more sophisticated models, but in ensuring the data that trains them meets the quality standards our applications demand. FiftyOne provides the tools to make this systematic dataset curation practical and scalable.

Try It Yourself

Ready to become a data detective in your own projects?

Get Started:

1. Install FiftyOne: `pip install fiftyone`
2. Access the complete Google Colab notebook at this link
3. Apply mistakenness and hardness analysis to your own datasets
4. Share your discoveries with the FiftyOne community

Next Steps:

- Apply these techniques to other classification challenges
- Explore FiftyOne Brain's other data quality metrics
- Integrate dataset curation into your MLOps workflows
- Contribute to the growing field of data-centric AI

The complete code is available as a Google Colab notebook here. Join the FiftyOne community and help advance the practice of systematic dataset quality assessment.